Expense Tracker with Smart Insighter

Process Compliance Document

Prepared by: **ARYAN BHARTI**
Date: April 2026
MP Online Internship Project

1. Introduction:

The Process Compliance Document defines the development practices, standards, and methodologies followed during the development of the **Expense Tracker with Smart Insighter**. The objective is to ensure that the project is developed in a structured, consistent, and quality-driven manner, similar to industry software development practices.

This document outlines how different stages of the Software Development Life Cycle (SDLC) were followed, ensuring proper planning, implementation, testing, and delivery of the system.

2. Development Methodology:

The project follows a **modular and iterative development approach**, inspired by Agile practices.

Instead of building the entire system at once, the development was carried out in smaller phases:

- Requirement understanding
- Feature-wise development
- Continuous testing
- Incremental improvements

This approach allowed:

- Faster development
- Early detection of errors
- Flexibility in adding new features

3. SDLC Phases Followed:

3.1 Requirement Analysis:

- Identified the need for an intelligent expense tracking system
- Defined key features such as expense management, categorization, and analytics
- Finalized scope and limitations

3.2 System Design:

- Designed system architecture (layered architecture)
- Defined database schema (Users and Expenses tables)
- Planned API structure and data flow

3.3 Implementation:

- Developed frontend using React
- Built backend APIs using FastAPI
- Integrated PostgreSQL database
- Implemented AI logic for categorization and predictions

3.4 Testing

- Performed unit, integration, and functional testing
- Validated API responses and UI interactions
- Ensured correctness of categorization and calculations

3.5 Deployment (Prototype Level)

- Application tested in local development environment
- Backend served using Uvicorn
- Frontend served using Vite development server

4. Coding Standards and Practices:

To maintain code quality and readability, the following practices were followed:

- Modular code structure (separate frontend, backend, services)
- Meaningful variable and function naming
- Proper indentation and formatting

- Reusable components in frontend
- Separation of business logic and API routes

These practices improve maintainability and collaboration.

5. Version Control and Collaboration:

Version control was managed using **Git and GitHub**.

Key practices:

- Regular commits with meaningful messages
- Version tracking for all changes
- Backup of source code
- Easy rollback in case of errors

6. API and Documentation Standards:

- RESTful API design principles were followed
- Standard HTTP methods used (GET, POST, PUT, DELETE)
- JSON used as data exchange format
- API documentation generated using Swagger UI

7. Testing Compliance:

The system was tested at multiple levels:

- Unit testing for individual components
- Integration testing for API communication
- Functional testing for feature validation
- Validation testing for input handling

Test cases were defined and executed to ensure reliability.

8. Security Compliance:

Basic security practices were implemented:

- Password hashing using bcrypt
- Input validation to prevent invalid data
- Authentication mechanism for user access
- Protected routes in frontend

9. Performance Considerations:

- Efficient API design to reduce response time
- Optimized database queries
- Lightweight frontend rendering using React

The system performs smoothly under normal usage conditions.

10. Risk Management:

Potential risks identified during development:

- Incorrect expense categorization
- Data inconsistency
- Invalid user inputs

Mitigation strategies:

- Validation checks
- Testing at multiple levels
- Error handling mechanisms

11. Limitations in Process Compliance:

- No formal CI/CD pipeline implemented

- Limited automation in testing
- No cloud deployment

These can be improved in future versions.

12. Conclusion:

The development of the Expense Tracker with Smart Insighter followed a structured and disciplined approach aligned with standard software engineering practices. From requirement analysis to testing, each phase of development was carried out systematically, ensuring a reliable and maintainable application.

The adoption of modular design, version control, testing strategies, and basic security practices demonstrates adherence to process compliance standards. Although the system is a prototype-level implementation, it reflects a strong foundation for transitioning into a production-level application with further enhancements.